

Legion — the story

What was built on the Legion machine, what it showed, and what to ask for

What this is

The Legion machine's work as a narrative, in five stops, for someone who has to **present it tomorrow**. Read in this order — each stop assumes the one before.

	Stop	Source	Lines
1	The map, and the one caveat	<code>README.md</code>	76
2	The session report — the core document	<code>session-2026-08-27-legion.md</code>	250
3	The headline result: neither route learns the task	<code>long-run-learning-2026-08-30.md</code>	168
4	The two integration routes, head to head	<code>comparison-isaacsim-vs-isaacclab.md</code>	112
5	The asks, grouped by who can act on them	<code>team-discussion_legion.md</code>	190

The three sentences to have ready

What was built: the RL toolkit now drives Martin's digital twin through two independent stacks on the GPU — bare Isaac Sim and Isaac Lab + Docker — and the Isaac Lab adapter is general, since it drove four Isaac Lab built-in tasks unchanged with observation dimensions from 4 to 60.

What was found: neither route learns to open the door, and the task **is not solvable as configured**. The handle sits 0.834 m from an arm with roughly 0.85 m of reach, the mobile base is not actuated, and every episode starts from the same pose. This was measured, not inferred.

What is needed: a decision from Martin on base actuation or start pose. Nothing else on the door task matters until the gripper can reach the handle.

The trap to avoid on stage

One route's reward **tripled** (105.8 → 334.3) and converged cleanly. That is real optimisation — of the approach terms, which are the ones that pay. Replayed deterministically, the trained policy moves the door in **0 of 6 episodes**. The 1.2% of training episodes where the door did move were exploration noise, and the recorded peak of exactly 3.1416 rad is the hinge's −180° hard stop, i.e. the door being *knocked* rather than pulled.

Presenting that curve as "learned to open the door" would be wrong in precisely the way the project brief's no-brute-force rule exists to prevent. **A rising reward curve is not evidence of task success** — logging door angle alongside reward is what made the diagnosis possible, and it is the transferable lesson of the whole round.

Two things to decide before you present

- **How to describe how this was produced.** The investigation, code and documents in this round were produced with an AI coding agent working under direction, over three days. Stop 2 raises this explicitly and leaves the decision open.
- **Whether the negative result leads or follows.** It is the most defensible thing here, and it is also the thing that sounds worst in a one-line summary. Leading with it and framing the plumbing as what made it findable is the stronger read.

Not included

`session-2026-08-28-gpu-fix-and-dt-wiring.md` (297 lines) — the live progress log, explicitly superseded by stop 2 and retained only because it records a wrong diagnosis being corrected. The supporting per-experiment detail and every measured number are in the Evidence packet; the two hands-on guides are in the Hands-on packet.

Generated 2026-09-02 from RL_DT@d827a53, which is the commit that pulled the Legion work onto the PNNL machine. Assembled verbatim from the markdown sources named in each section header; nothing was rewritten for this packet, and the framing notes under each header are the only added text. Chromium print does not emit page numbers, so the section banners are the navigation.

Stop 1 of 5 — the map, and the one caveat

RL_DT/_docs_legion/README.md - 77 lines

Two pages. Read its closing section 'The one thing to keep in mind' twice — it is the claim most easily lost when skimming, and the one most easily overclaimed in a meeting.

_docs_legion — documentation from the Legion machine

Everything here was produced on `trossen-ai` (the Legion laptop, RTX 5090) during **2026-08-27 → 08-31**. It is kept separate from `../_docs/`, which holds the original documentation written on Dewei's laptop and the PNNL workstation.

The two folders are not duplicates. They describe different machines with genuinely different capabilities — this one has a Blackwell GPU and two Isaac Sim installations, but cannot reach tanuki or the PNNL shared drives. Where a `_legion` file appears to contradict its counterpart in `_docs/`, both are correct about their own machine.

Nothing in `../_docs/` was modified.

Start here

If you want	Read
The overall story, for a team audience	<code>session-2026-08-27-legion.md</code>
To play with the digital twin yourself, in the GUI	<code>ISAAC_SIM_UI_GUIDE.md</code>
To run the twin through Docker	<code>DIGITAL_TWIN_HOWTO.md</code>
Whether any of it actually learns	<code>long-run-learning-2026-08-30.md</code>
A specific measured number or bug	<code>PUBLICATION_MATERIALS_legion.md</code>
What to tell Malachi / Martin / Alvika	<code>team-discussion_legion.md</code>

Everything, by kind

Live — mirrors of the originals, for this machine

File	Mirrors	Holds
<code>WORKLOG_legion.md</code>	<code>WORKLOG.md</code>	Round index and entry
<code>PUBLICATION_MATERIALS_legion.md</code>	<code>PUBLICATION_MATERIALS.md</code>	6 results, 11 code defects, 2 method notes — the single home for every number
<code>decisions_legion.md</code>	<code>decisions.md</code>	9 decisions, each with what it rules out
<code>environment_legion.md</code>	<code>environment.md</code>	Machine, kernel pin, network limits, both Isaac Sim installs
<code>team-discussion_legion.md</code>	<code>team-discussion.md</code>	Grouped by who can act

History

- `rounds/Round-2026-08-27-legion-gpu-fix-and-dt-integration.md` — day-by-day narrative, including the four mistakes made and what they cost.

Reports and guides

File	What it is
<code>session-2026-08-27-legion.md</code>	The session report, written to be presented
<code>session-2026-08-28-gpu-fix-and-dt-wiring.md</code>	The live progress log kept during the work — superseded by the report above, retained because it records a wrong diagnosis being corrected
<code>ISAAC_SIM_UI_GUIDE.md</code>	Exploring the twin in the Isaac Sim GUI — no Docker, no RL
<code>DIGITAL_TWIN_HOWTO.md</code>	Running the twin through the Docker container
<code>comparison-isaacsim-vs-isaacclab.md</code>	The two integration routes, head to head
<code>long-run-learning-2026-08-30.md</code>	The multi-hour learning runs, why neither route solves the task, and the four problems found doing them
<code>gpu-retest-2026-08-29.md</code>	The re-tested Gymnasium environments in detail
<code>isaacclab-builtin-tests-2026-08-29.md</code>	Toolkit against four Isaac Lab built-in tasks
<code>dt-via-isaacclab-2026-08-29.md</code>	The 1→128 environment scaling sweep

The one thing to keep in mind when reading any of this

Neither route learns the door task, and the task is not solvable as configured. The toolkit drives the digital twin through two independent stacks and that plumbing is verified in detail — but multi-hour runs showed the trained policy never opens the door, because the handle sits 0.834 m from an arm with roughly 0.85 m of reach and the mobile base is not actuated. Reward tripling on one route is real optimisation of the approach terms, not progress on the task. The full account, with the measurements, is in `long-run-learning-2026-08-30.md`. Repeated here because it is the easiest thing to lose when skimming, and the easiest thing to overclaim.

Merging this back

These files were deliberately kept parallel rather than merged into `../_docs/`, so that work from two machines would not be interleaved before anyone had reviewed it. Once the two are reconciled, most of this belongs folded into the originals — `../_docs/README.md` sets a budget of six live files and warns against sprawl, and this folder adds fourteen more. The natural merge is: numbers into `PUBLICATION_MATERIALS.md`, decisions appended to `decisions.md`, the machine section into `environment.md`, and the guides kept as-is since they have no counterpart.

Stop 2 of 5 — the session report

RL_DT/_docs_legion/session-2026-08-27-legion.md - 251 lines

The core document, and the one written to be presented. Sections 1-7 are the spine of any talk. Note the honest-note block in section 1: a wrong diagnosis was recorded rather than quietly corrected.

Connecting the RL toolkit to the digital twin — session report

Machine: trossen-ai (Legion, RTX 5090) · **Dates:** 2026-08-27 → 2026-08-29 **Repo:** github.com/Dewei-Wang-xx/RL4DT-Toolkit · **Detail:** WORKLOG_legion.md

Summary

The RL toolkit now drives the digital twin, through **two independent stacks**, on the GPU. Getting there required fixing a bug that prevented the toolkit from running on this hardware at all — and that fix incidentally cleared a blocker the project had already documented as standing in the way of container-based Isaac work.

Six things that were not true three days ago:

1. The toolkit runs on Blackwell GPUs (it segfaulted; now trains, and actually uses the GPU rather than silently falling back to CPU).
2. The toolkit imports in a torch-only container — the blocker recorded in `strategy.md` Part 2.
3. The toolkit drives Martin's door task through **Isaac Lab**, scaling to 128 parallel robots.
4. The toolkit drives the same task through **bare Isaac Sim**, with no Isaac Lab and no Docker.
5. The adapter is demonstrably general — it drove four Isaac Lab built-in tasks unchanged.
6. Isaac Sim is installed two ways, including a working GUI for exploring the twin visually.

What is now known, and was an open question when this was first written: multi-hour runs were subsequently done on both routes. **Neither learns to open the door**, and the reason is measured - the handle is out of reach with the actions the task exposes. That is arguably the most useful result of the whole exercise. See `long-run-learning-2026-08-30.md` and Limits.

1. The blocker: the toolkit could not run on this machine

`TorchTD3-v0` segfaulted a few milliseconds after starting, before a single training step.

Root cause: TensorFlow and Triton (PyTorch's GPU compiler, loaded lazily when an optimizer is built) ship conflicting native libraries. **Whichever loads second crashes.** Two lines reproduce it:

```
import tensorflow          # then
import triton._C.libtriton # -> SIGSEGV
```

Reverse the order and both load fine. The toolkit's import order loaded TensorFlow first — through the Keras agents, and through `torch.utils.tensorboard`, which imports TF — so Triton lost.

A second symptom is arguably worse, because it is silent. Importing TensorFlow installs CUDA stub libraries, after which `torch.cuda.is_available()` returns `False`. Any torch agent in a process that has imported TF has been **running on CPU while reporting nothing wrong** — including, presumably, on machines where the segfault never occurs.

Fix: 5 files, 191 lines. Pre-load Triton at package import, guarded so it does nothing when TF is already present; make the Keras agents, models and the TF-backed `Model` base class import lazily. Additive — the Keras agents were regression-tested and still train. Standalone patch and full write-up in `../_patches/README.md`.

The side effect was worth more than the fix itself. The lazy imports mean the package now imports where TensorFlow is not installed at all — which is exactly the Isaac Lab container. Everything in sections 3–6 depends on this.

An honest note on how this was found. The crash was first diagnosed as a known upstream PyTorch/Blackwell bug and written up as unfixable locally. That was wrong. It was corrected only because Dewei pointed out the same code runs fine on other machines, which prompted a proper bisect. Recorded because the wrong turn is instructive.

Three further toolkit bugs surfaced along the way, all pre-existing and all with fixes ready: `setup.py` omits every subpackage so `pip install -e .` is broken; three subpackages lack `__init__.py`; `gymnasium-robotics` is required but undeclared. Write-ups for Malachi in `team-discussion_legion.md`.

2. Environments re-tested on GPU

Four of five published environments reproduced, all reporting Device: `cuda`.

Environment	Episodes × steps	Final avg-20	Wall time
Pendulum-v1	100 × 200	−160.30	111 s
MountainCarContinuous-v0	100 × 999	−1.02	477 s
Hopper-v5	300 × 1000	187.10	71 s
AdroitHandDoor-v1	20 × 200	−42.31	23 s
FetchReachDense-Flat-v0	500 × 50	could not start	—

A dedicated Pendulum run crosses avg-20 of −200 at **episode 61** and ends at **−132.3**.

Two caveats that matter more than the numbers. `FetchReach` fails inside `gymnasium-robotics`' own setup code against `mujoco 3.12.0` — before the toolkit is involved, and Fetch-specific. And `AdroitHandDoor`'s reward function changed in `gymnasium-robotics 1.2.1` *without an environment version bump*, so that figure is not comparable to the published one. More generally these results are **not** comparable to the published ones: those were CPU runs on a different machine, and the driver seeds from wall-clock time, so nothing is reproducible either way.

3. Isaac Sim installed two ways, deliberately

	Purpose	Status
Docker container (Isaac Lab 2.3.2 + Isaac Sim 5.1.0)	Training	Built locally, 34.7 GB, GPU passthrough verified
Workstation install (Isaac Sim 5.1.0)	Looking at the twin	GUI reaches "app ready" in ~16 s, no errors

Version 5.1.0 in both, per Alvika's warning — the download page defaults to 6.0.1, and Martin's assets were authored in 5.1. A 5.0.0 build had been fetched from a PNNL machine first; 5.1.0 turned out to download directly here without an NGC login, so no transfer was needed.

Two guides written: `ISAAC_SIM_UI_GUIDE.md` for exploring the twin visually, and `DIGITAL_TWIN_HOWTO.md` for the Docker route. Both mark which steps were verified and which were not.

4. Route A — digital twin via Isaac Lab + Docker

Martin's task as written, driven by the toolkit's TD3 through a vectorised adapter.

--num_envs	Env-steps/s	Speed-up
1	15–17	baseline
16	180–191	~11×
64	134–419	~9–25×
128	181–271	~12–17×

Throughput climbs steeply to 16 environments then flattens, because action selection is still **one forward pass per environment** in a Python loop while the simulator steps them all together on the GPU.

Why parallelism is a correctness issue here, not just speed. `warmup_size` is 2500 — TD3 takes random actions until 2500 transitions are buffered. At one environment a 40-step episode contributes 40 transitions; at 128 it contributes 5120. **A single-environment configuration cannot practically reach the point where learning begins.**

5. Route B — digital twin via bare Isaac Sim

No Isaac Lab, no Docker. `door_env_direct.py` (345 lines) reimplements what Isaac Lab was providing: scene loading, observation assembly, action application, reward, termination, reset.

Works: **35 observations, 7 actions, on cuda, 130–200 steps/s.**

Verified against physics rather than inferred from absence of errors — all eight prim paths resolve, the articulation exposes 15 DOF with expected names, and commanding the door hinge to $-30^\circ/-90^\circ/-170^\circ$ reads back within 2° .

That check caught a real bug: the door opens in the *negative* direction, so the first implementation scored a fully-opened door as zero reward.

Scope, stated plainly: the reward here is three terms (approach / opening / effort) against Martin's roughly ten tuned terms across 1140 lines. A structurally equivalent task, **not** a reproduction.

6. The adapter generalises

Four Isaac Lab built-in tasks, same adapter, **no per-task code:**

Task	Obs	Actions	Throughput
Isaac-Cartpole-v0	4	1	659 env-steps/s
Isaac-Ant-v0	60	8	440 env-steps/s
Isaac-Reach-Franka-v0	32	7	257 env-steps/s
Isaac-Open-Drawer-Franka-v0	31	8	364 env-steps/s

Observation dimensions from 4 to 60. This is what shows the adapter is a genuine toolkit↔Isaac Lab bridge rather than something shaped around one scene. `Open-Drawer-Franka` is NVIDIA's cabinet task — structurally the same problem as Martin's door, and probably what it was modelled on.

7. Comparing the two routes

	Bare Isaac Sim	Isaac Lab + Docker
Throughput, 1 env	148–151 steps/s	15–17 steps/s
Best measured	148–151 (1 env only)	~419 (64 envs)
Parallel environments	1	1→128 tested
Code written	345 lines	248 lines
Task definition reused	none	1140 lines (Martin's)
Disk	18 GB	34.7 GB image + 23 GB base
Runs without Docker	yes	no
GUI	yes, verified	awkward from container

The single-environment gap flatters the direct route: that overhead is Isaac Lab doing the work that makes many environments possible, and the direct route cannot scale without reimplementing it.

Recommendation — keep both, for different jobs. Isaac Lab for training: it has the parallelism, Martin's real task, and a reproducible pinned environment, and it is the only route that can reach TD3's warmup threshold. Bare Isaac Sim for GUI exploration, for learning how the twin is put together, and as the natural starting point if the real Ridgeback + UR5e becomes the target — its plain step loop maps onto one physical robot far more directly than a vectorised manager-based framework does.

What this does not show

Worth stating before any of this is presented, because it is easy to lose:

- **Neither route learns the door task.** Multi-hour runs settled this: the direct route's reward tripled and converged, but the trained policy replayed deterministically moves the door in 0 of 6 episodes. The Isaac Lab route never moved the door at all. The task is not solvable with the actions currently exposed - the handle is 0.834 m from an arm with ~0.85 m of reach, and the base is not actuated. Full account in `long-run-learning-2026-08-30.md`.
- **The two routes do not use the same reward**, so they cannot be compared on learning at all.
- **Throughput figures are order-of-magnitude.** Two episodes per configuration, with large per-episode variance (419 then 134 env-steps/s at 64 environments).
- **Nothing is reproducible run to run** — the driver seeds from wall-clock time.

The natural next question — *does TD3 actually learn to open the door, and how does it compare to the existing `rl_games` PPO baseline?* — is unanswered, and needs long runs on the Isaac Lab route.

Next steps

1. **Resolve the reachability question with Martin.** Nothing else matters until the gripper can reach the handle: either actuate the base in his task, or park the robot closer. This is the blocker, and it is measured rather than suspected.
2. **`batch_action` on the agent.** The highest-value toolkit change: one batched forward pass for N states instead of N separate passes. It is what caps throughput, and unlike the adapters it is a change to Malachi's code, so it needs his view.

3. **Fix the $O(n)$ replay-buffer sampling upstream** — it throttles every long run in the toolkit, not just the Isaac ones.
4. **Send the three toolkit bugs upstream** — write-ups ready in `team-discussion_legion.md`.
5. **Decide `test_torch_td3.py`'s device handling** (Malachi's call — 12 tests assume CPU tensors).
6. **Revisit the kernel pin** when Ubuntu ships a matching NVIDIA module, so the machine resumes kernel security updates.

Where everything is

Session detail, day by day	<code>rounds/Round-2026-08-27-legion-gpu-fix-and-dt-integration.md</code>
Every number and code defect	<code>PUBLICATION_MATERIALS_legion.md</code>
Machine setup and its limits	<code>environment_legion.md</code>
Decisions and their reasoning	<code>decisions_legion.md</code>
Items for Malachi, Martin, Alvika	<code>team-discussion_legion.md</code>
The toolkit fix, standalone	<code>../_patches/README.md</code>
Isaac Lab route code	<code>../_toolkits/toolkit-isaacclab/isaac_integration/</code>
Bare Isaac Sim route code	<code>../_toolkits/toolkit-isaacsim/</code>

A note on how this work was produced: the investigation, code and documents in this round were produced with an AI coding agent working under direction, over three days. Worth deciding how you want that described before the material is presented onward.

Stop 3 of 5 — the headline result

RL_DT/_docs_legion/long-run-learning-2026-08-30.md - 169 lines

Written after stop 2 and it supersedes stop 2's open question. This is the most defensible result of the round: the task is not solvable as configured, measured rather than inferred.

Do the two routes actually learn? — multi-hour runs, 2026-08-30/31

The question left open by every earlier result: the toolkit *drives* the digital twin through two stacks, but does anything **learn to open the door**?

Answer: no. Neither route learned the task, and we now know why. The reason turned out to be worth more than a learning curve would have been.

Headline

	Bare Isaac Sim (base actuated)	Isaac Lab + Docker (Martin's config)
Episodes	~1,610	~210 × 64 envs
avg-20 reward	105.8 → 334.3 (peak 338.4)	5.85 → 9.38
Plateaued at	~episode 400	immediately
Door ever opened	2 of 162 sampled episodes (1.2%)	never
Trained policy replayed	door never moves, 6/6 episodes	—

The direct route's reward **tripled**, which looks like learning and is — but not of the task. It learned to approach and align with the handle, which is what the un-gated reward terms pay for. The door-opening terms contributed essentially nothing.

The decisive test

A rising reward curve is not evidence of task success. So the trained policy was reloaded and replayed deterministically (`train=False`, no exploration noise), six episodes:

```
ep0: door never moved
ep1: door never moved
ep2: door never moved
ep3: door never moved
ep4: door never moved
ep5: door never moved
```

The 2 door-openings during training were exploration noise, not policy. TD3 adds random noise to actions while training; occasionally that noise shoved the door. The peak recorded — exactly 3.1416 rad, i.e. precisely the hinge's -180° limit — is the signature of the door being knocked to its stop rather than pulled open.

This matters beyond bookkeeping: the brief forbids **brute-force pushing**. Had this been reported as "learned to open the door", it would have been both wrong and wrong in the specific way the project is meant to avoid. Evidence:

`_toolkits/toolkit-isaacsim/_isaacsim_notes/policy-replay-verdict.txt`.

Why the Isaac Lab route was flat

Not an algorithm problem — a **geometry** problem, measured before the runs:

```
handle -> arm base distance          0.834 m    (UR5e reach ~0.85 m)
closest gripper->handle, 300 random arm poses 0.207 m
closest, 400 poses WITH base translation  0.137 m
```

Martin's `ActionsCfg` exposes `arm_action` and `gripper_action` only — the Ridgeback's three base DOF are not actuated — and `EventCfg` uses `position_range: (0.0, 0.0)`, so every episode starts from the same out-of-reach pose. **The gripper cannot get within 20 cm of the handle**, so no policy can open the door, and the flat 9.38 reward is the correct outcome rather than a failure to train.

Its replay buffer filled to capacity (1,000,000) with transitions that contain no successful behaviour to imitate.

The direct route was given the three base joints (action dim 7 $\rightarrow$ 10) precisely because of this measurement. That is why it could improve at all — and it still did not solve the task.

What went wrong along the way

Four problems were found and fixed during these runs. Three were mine; one is upstream and affects everyone.

1. Replay-buffer sampling was $O(n)$ — upstream, affects all long runs

`er.py` sampled with `np.random.choice(max_index, size=256, replace=False)`. NumPy implements that by building and permuting an array of the **whole buffer** on every call:

buffer	choice(replace=False)	randint	penalty
1,000	0.40 ms	0.03 ms	12×
50,000	4.80 ms	0.04 ms	112×
200,000	19.88 ms	0.04 ms	460×

Caught 1.5 h into the first attempt: the direct route had fallen from 150 to 10.8 steps/s at buffer 56k, the Isaac Lab route from 195 to 21 env-steps/s, with **the GPU idling at 3% while the CPU pegged** — sampling, not physics or the network, was the bottleneck. Switched to `np.random.randint` (sampling with replacement, as SB3, CleanRL and the reference TD3 implementation all do). Measured 5–6 ms $\rightarrow$ 0.050 ms at 60k entries.

Not Isaac-specific. This throttles every sufficiently long run in the toolkit, including the published Gymnasium results. Short runs never fill the buffer enough to notice.

2. Half the ported reward was silently dead — mine

`left_inner_finger` and `right_inner_finger` report the **same world transform as wrist_3_link** — verified through `XFormPrim`, `RigidPrim`, and USD's own `ComputeLocalToWorldTransform`, and confirmed to move with the arm but never to separate from the wrist. Martin's straddle test is `lfinger_z > handle_z AND rfinger_z < handle_z`; with identical values that is `x > h AND x < h`, false for every possible input.

That zeroed `align_grasp_around_handle` and `approach_gripper_handle` — and because the three opening terms are **gated on the grasp state**, every door-opening term was permanently zero too. The first overnight run had *no gradient toward opening the door at all*. Replaced with a TCP-height proxy that actually varies with behaviour.

3. Neither training script logged reward — mine

The agent writes its own critic/actor losses, but reward logging lives in the toolkit's driver, which both integration scripts bypass with their own loops. An 8-hour run would have produced loss curves and **no learning curve**. Caught by a 12-episode pilot before committing the night. Both scripts now log episode reward, a 20-episode average, door angle (per-episode and best), buffer size and throughput.

Logging **door angle** rather than only reward is what made the final diagnosis possible: reward rose while door angle stayed at zero, which is exactly the signature of optimising the shaped terms without solving the task.

4. A `.gitignore` pattern hid a source directory — mine

`_toolkits/**/buffers/`, intended for replay-buffer `.npy` dumps under `results/`, also matched `jlab_opt_control/buffers/` — the source package. The buffer fix would not commit, and the package had never been in git at all. Same class of mistake as an earlier `rsync --exclude='buffers'` that deleted that same directory: a pattern aimed at data that also caught code.

What this establishes

Established: - The toolkit trains against the digital twin through both stacks, on GPU, for hours, unattended. - Reward optimisation works — the direct route tripled its reward and converged cleanly. - The door task **as currently configured is not solvable**, and the reason is measured, not guessed. - Two real toolkit bugs found (buffer sampling; and see `PUBLICATION_MATERIALS_legion.md` for the packaging and TF/triton ones).

Not established: - That TD3 can learn this task. It has not been shown to, on either route. - Any comparison of *learning* between the routes. They differ in action space (7 vs 10), observation vector (43 vs 35), and reward fidelity (13 terms vs 8, two of which were broken for the first run).

What would need to change before trying again

1. **Make the handle reachable.** Either actuate the base in Martin's task, or move the robot's start pose within arm's reach. This is the blocker; nothing else matters until it is fixed, and it is a question for Martin.
2. **Fix the fingertip transforms**, or agree a substitute. The grasp-detection chain depends on them, and the opening rewards are gated behind it.
3. **Randomise the start pose.** `position_range` is `(0.0, 0.0)`; every episode is identical, so there is nothing to generalise over — already on the project roadmap.
4. **Expect a long horizon.** Approach → align → straddle → grasp → pull 180° is a deep chain for TD3 from sparse gated rewards. Even correctly configured, this may need curriculum, demonstrations, or a different algorithm. The existing `rl_games` PPO baseline is the natural comparison and has not been run here.
5. **Batched action selection** (`batch_action` on the agent) before any serious scaling — still the highest-value toolkit change.

Artifacts

Direct-route training log	<code>_toolkits/toolkit-isaacsim/_isaacsim_notes/run3-training-log.txt</code>
Isaac Lab training log	<code>_toolkits/toolkit-isaacsim/_isaacsim_notes/run3-training-log.txt</code>
Policy replay verdict	<code>_toolkits/toolkit-isaacsim/_isaacsim_notes/policy-replay-verdict.txt</code>
Reachability measurements	<code>_toolkits/toolkit-isaacsim/_isaacsim_notes/reachability-*.txt</code>
Fingertip prim evidence	<code>_toolkits/toolkit-isaacsim/_isaacsim_notes/fingertip-prims-broken.txt</code>

TensorBoard event files (144 MB and 701 MB) are **not** committed; the curves are summarised above.

Stop 4 of 5 — the two routes, head to head

RL_DT/_docs_legion/comparison-isaacsim-vs-isaacsimlab.md - 113 lines

Read its opening 'Read this before the numbers' first. The recommendation is keep both for different jobs — Isaac Lab to train, bare Isaac Sim to look and to map onto real hardware.

Two ways to connect the toolkit to the digital twin — a comparison

2026-08-29. Both routes work. This is what each costs and what each buys.

- **Route A — bare Isaac Sim.** `_toolkits/toolkit-isaacsim/`. Isaac Sim's Python API only.
- **Route B — Isaac Lab + Docker.** `_toolkits/toolkit-isaacsimlab/`. Alvika's container, Martin's task.

Read this before the numbers

The two routes are not solving the same task. Route B runs Martin's task as written: ~10 shaped reward terms across 1140 lines, tuned together. Route A runs a **simplified task I wrote** — same robot, same door joint, same 6+1 actions, but a three-term reward (approach / opening / effort).

So this comparison is valid for **throughput, code size, dependencies and operational cost**. It is **not** valid for learning performance, and no conclusion of the form "route X learns better" can be drawn from anything here. Neither route has been shown to learn the door task at all.

Head to head

Matched conditions: single environment, 2 episodes × 40 steps, RTX 5090, TorchTD3 on `cuda`.

	Route A (bare Isaac Sim)	Route B (Isaac Lab + Docker)
Throughput, 1 env	148–151 steps/s	15–17 steps/s
Throughput, best measured	148–151 (1 env only)	~419 env-steps/s (64 envs)
Parallel environments	1	1 → 128 tested
Observation dim	35 (mine)	43 (Martin's)
Action dim	7	7
Code I had to write	345 lines	248 lines
Task definition depended on	none — written from scratch	1140 lines (Martin's, reused)
Disk footprint	18 GB	34.7 GB image (+23 GB base)
Startup to first step	~15 s	~20 s + container
Runs without Docker	yes	no
GUI available	yes, verified	awkward from container

The single-environment number is misleading on its own

Route A looks ~10× faster per step at 1 environment — but that gap is mostly Isaac Lab's per-step management overhead, which exists precisely because it is doing the work that makes *many* environments possible. Route B at 64 environments reaches ~419 env-steps/s, roughly 3× Route A's best, and it can go further. **Route A cannot be scaled without writing the parallelism myself**, which is the single largest piece of work Isaac Lab is doing.

Why that matters more than it appears

warmup_size is 2500: TD3 collects random transitions and does not train until the buffer holds that many. At 1 environment, a 40-step episode contributes 40 transitions. **Route A as it stands cannot practically reach the start of learning.** Route B at 128 environments contributes 5120 per episode and clears warmup immediately.

For *validating plumbing*, single-environment is fine. For *training anything*, it is not.

Where each route's effort goes

Route A's 345 lines are not equivalent to Route B's 248. Route B's 248 lines are glue; the actual task — scene, observations, ten reward terms, resets, terminations, randomisation — comes from Martin's 1140 lines, already written and debugged. Route A has no such foundation: my 345 lines implement a *far simpler* task and would need to grow substantially to match, plus everything Isaac Lab provides for free (parallel envs, config-driven task composition, the rl_games/RSL-RL/skrl integrations, tooling).

Realistically, matching Martin's task through Route A means reimplementing most of those 1140 lines and then the parallelism on top.

What Route A genuinely buys

1. **No Docker.** Runs directly. Simpler mental model, fewer moving parts, no mounts.
2. **A working GUI.** Verified: launches in ~16 s, RTX renderer active. This is the better route for *looking at* the twin, which is a real need distinct from training.
3. **Full control, no framework conventions.** If Isaac Lab's manager abstraction ever fights a requirement, this route has no such constraint.
4. **Closer to a hardware-style control loop.** Worth remembering if the real Ridgeback + UR5e is the eventual target: Isaac Lab's vectorised, manager-based structure does not map onto one physical robot as directly as a plain step loop does.

What Route B genuinely buys

1. **Parallelism**, which is the whole reason to use Isaac Sim for RL, and the thing Route A lacks.
2. **Martin's task, as written** — 1140 lines of tuned reward shaping reused rather than reinvented.
3. **A reproducible environment.** Version-pinned in `.env.base`; the same image runs for everyone.
4. **A generalising bridge.** The same adapter drove four built-in Isaac Lab tasks unchanged (obs 4→60, actions 1→8) — see `isaacclab-builtin-tests-2026-08-29.md`.

Recommendation

Keep both, for different jobs. They are not competing.

- **Route B for training.** It has the parallelism, the real task, and the reproducible environment. It is the only one of the two that can currently reach the start of learning.
- **Route A for looking, learning and experimenting.** GUI exploration, understanding the scene, trying ideas without container overhead, and as the natural starting point if real hardware becomes the target.

The strongest argument against making Route A the primary path is not throughput — it is that matching Martin's task means rewriting ~1140 lines of working, tuned code, and then writing the parallelism Isaac Lab already provides.

The one change that would help both

Action selection is one forward pass per environment, in a Python loop. Route B's throughput plateaus past ~16 environments because of it, and Route A would hit the same wall the moment it gained parallelism. A `batch_action` method on the agent — one batched forward pass for N states — is the highest-value next change, and it is a change to the toolkit itself rather than to either adapter.

Caveats on these measurements

- 2 episodes per configuration. Per-episode variance was large (419 then 134 env-steps/s at 64 envs), so treat throughput as order-of-magnitude.
- The driver seeds from wall-clock time, so nothing here is reproducible run to run.
- Route A's observation vector (35) differs from Route B's (43) because I assembled it myself; they are not the same observation.
- No learning was measured on either route.

Stop 5 of 5 — the asks, grouped by who can act

RL_DT/_docs_legion/team-discussion_legion.md - 191 lines

The only file with actions attached: three items for Malachi, a blocking question for Martin, three notes for Alvika. STALE: its closing 'Team-wide' section predates stop 3 and still says every run was minutes long. Stop 3 wins.

Team discussion — items arising from the Legion machine work

Companion to `team-discussion.md`, which is **not modified**. Grouped by who can act on each item. Every number cited lives in `PUBLICATION_MATERIALS_legion.md`.

For Malachi — three toolkit bugs and one proposal

1. TensorFlow and Triton conflict, and it segfaults the torch agents

The most consequential finding. `TorchTD3-v0` could not run at all on this machine — a segfault before the first training step. Root cause is a native-library load-order conflict between TensorFlow and Triton (which torch pulls in when constructing an optimizer): **whichever loads second crashes**. Full write-up and the two-line reproduction: D1.

A second symptom is arguably worse than the crash, because it is silent: importing TensorFlow installs CUDA stubs that make `torch.cuda.is_available()` return `False`. **Any torch agent in a process that has imported TF has been running on CPU while reporting nothing wrong** — including, presumably, on other machines where the segfault does not occur.

The fix is 5 files / 191 lines, standalone in `_patches/toolkit-tf-triton-fix.patch`, and is additive: the Keras agents were regression-tested and still train. It also clears the blocker recorded in `strategy.md` Part 2 — the package now imports in a torch-only container, which is what made the Isaac Lab work possible.

Ask: is this worth upstreaming, and if so does the guarded pre-load approach look right to you?

2. `pip install -e .` does not work — packaging omits every subpackage

`setup.py` declares `packages=['jlab_opt_control']` with no subpackages, and `buffers/`, `drivers/` and `utils/` have no `__init__.py`. A strict editable install therefore exposes only the top-level package and `import jlab_opt_control.buffers` fails. It goes unnoticed because running from the repo root finds the subpackages via the current directory. D2, D3. Small, safe fix — `find_packages()` plus three `__init__.py` files.

3. `gymnasium-robotics` is required but not declared

`envs/robotics_flat.py` needs it, and `utests/test_registry.py::test_env` fails without it, but it is in neither `requirements.txt` nor `env.yaml`. Same category as the `pysindy` / `scikit-learn` gap already noted in `environment.md`. D4 — though see D6 before choosing a version to pin.

3b. Replay-buffer sampling is $O(n)$ — this affects your published results too

`er.py` samples with `np.random.choice(max_index, size=256, replace=False)`. NumPy implements that by permuting an array of the entire buffer on every call:

buffer	<code>choice(replace=False)</code>	<code>randint</code>	penalty
50,000	4.80 ms	0.04 ms	112x
200,000	19.88 ms	0.04 ms	460x

We hit it live: throughput collapsed from 150 to 10.8 steps/s with the **GPU idling at 3%** while the CPU pegged - the sampling was the bottleneck, not the physics or the network. One-line fix to `np.random.randint` (sampling with replacement, as SB3, CleanRL and the reference TD3 implementation all do): 5-6 ms -> 0.050 ms at 60k entries.

This is not Isaac-specific. Any run long enough to fill a buffer is affected, including the published Gymnasium results - short runs simply never reach the buffer sizes where it bites. D9.

4. Proposal — a `batch_action` method on the agent

The highest-value next change for either integration route. Action selection is currently one forward pass per environment, in a Python loop. That is why throughput plateaus past ~16 environments (R3) despite the simulator stepping all of them together on the GPU. One batched forward pass for N states would remove it. Unlike everything else done here, this is a change to the agent itself rather than to a wrapper, so it needs your view on the interface.

5. Decision needed — `test_torch_td3.py` and the GPU

With the TF/Triton fix in place, 12 tests fail with a CPU/CUDA device mismatch: they build plain CPU tensors while the agent's networks now correctly live on the GPU. **Not a regression** — on the unmodified toolkit those tests *segfault* on this machine, and forcing `"device": "cpu"` makes all 47 pass. Deliberately left unfixed, because there are two defensible options and it is your test suite: make the tests device-aware (keeps GPU coverage) or pin them to CPU (smaller change, GPU path goes untested). D5.

For Martin — a blocking question about the task, then two asset issues

0. The handle may not be reachable in your task either — worth checking

The most important item here. Measured on the door scene:

```
handle -> arm base distance           0.834 m   (UR5e reach ~0.85 m)
closest gripper->handle, 300 random arm poses  0.207 m
closest, 400 poses WITH base translation      0.137 m
```

`ActionsCfg` exposes `arm_action` and `gripper_action` only, so the Ridgeback's three base DOF are never actuated, and `EventCfg` sets `position_range: (0.0, 0.0)` so every episode starts from the same pose. On our runs the gripper never got within 20 cm of the handle and the door never moved - across 210 episodes x 64 environments.

Question: is the base meant to be actuated, or the robot meant to be parked closer? Either resolves it. We added the base joints on our bare-Isaac-Sim variant to work around it, but that is a divergence from your config rather than a fix to it. Worth checking whether your own `r1_games` PPO runs ever open the door - if they do, something in our setup differs and we would like to know what.

0b. Do your fingertip reward terms fire?

`left_inner_finger` and `right_inner_finger` report the same world transform as `wrist_3_link` when read through `XFormPrim`, `RigidPrim`, or USD's `ComputeLocalToWorldTransform`. Your `align_grasp_around_handle` and `approach_gripper_handle` test whether the two fingertips straddle the handle in z, which with identical values can never be true - and the three opening terms are gated on that grasp state.

Your version goes through Isaac Lab's `FrameTransformer` rather than raw prims, so it may resolve correctly where ours did not - **we did not test that**. But if it does not, several of your reward terms are dead and the opening terms with them. D11.

Martin — two asset issues, one trivial and one cosmetic

1. The door scene is referenced by bare filename

RL_CloudTesting/ur5e_door_open_env_cfg.py had `USD_PATH = "TrainingScene_flattened.usd"`, but the file lives in `ENV_OT_UR_door/`, one directory across. Isaac Lab fails with `FileNotFoundError` until it is changed to `"../ENV_OT_UR_door/TrainingScene_flattened.usd"`. The line sits under a "User-editable paths" comment, so this may be intentional per-setup configuration rather than a bug — worth confirming which. D7.

2. A stale absolute Windows path inside the flattened USD

Every load prints a warning about not finding `C:/Users/prat615/.../OpentronsDoorAdapter_large.usdc`. It is harmless — the handle geometry was baked in when the scene was flattened, and the prim the reward depends on (`/World/OpenTronsFlex/OpentronsDoorAdapter_large/LooRoll/Handle`) is present and valid, mesh and revolute joint included. It caused a false alarm here before being checked properly. Worth clearing from the asset if it is easy, purely so it stops looking alarming. D8.

3. Question — is the door meant to open negatively?

The hinge's USD limits are `lowerLimit=-180°`, `upperLimit=0°`, so the door swings negative as it opens. Martin's own config carries the same fact as `DOOR_OPEN_DIR`, so this looks intended — but it is the kind of sign convention worth stating explicitly somewhere, since getting it backwards scores a correctly-opened door as zero reward (which happened here on the first attempt).

For Alvika — container and version notes

1. The 5.1.0 warning was well placed, and worth making louder

The Isaac Sim download page defaults to 6.0.1; a 5.0.0 build was downloaded here by mistake before the version was checked. Worth stating the required version in the `isaacsim-dt-main` README itself rather than only in conversation, since `.env.base` already pins `ISAACSIM_VERSION=5.1.0` and the two should visibly agree.

Also useful to know: Isaac Sim 5.1.0 downloads directly from `download.isaacsim.omniverse.nvidia.com` with no NGC login, as does the container base image from `nvcr.io`. Neither needed credentials, which matters for machines that cannot reach GitHub.

2. `envs_builtin/` does not exist in this copy

The README documents baking small scenes into the image via `envs_builtin/`, but the directory is absent from the copy here, so nothing is baked in. Mounting Martin's assets externally worked fine and is probably the better fit at 2 GB — but the README and the repo currently disagree.

3. `.env.base` ships with another machine's paths

`EXTERNAL_USD_PATH` and `EXTERNAL_PROJECT_PATH` pointed at `/home/aenabledrobotics2/Desktop/...`. Expected for a per-machine setting, but a placeholder value plus a line in the README ("set these two before first start") would save the next person a confusing empty mount.

Team-wide — what these results do and do not show

Worth being explicit before any of this is presented, because the distinction is easy to lose:

Established: the toolkit can drive Martin's digital twin, through two independent routes; the adapter generalises across four Isaac Lab built-in tasks with observation dimensions from 4 to 60; parallelism scales to at least 128 environments.

Not established: that any of this *learns the door task*. Every run was minutes long with an essentially untrained policy. The two routes do not even use the same reward function — the direct route uses a simplified three-term reward written here, not Martin's ten tuned terms. No claim about learning performance is supported by any number in these documents.

The natural next question — *does TD3 actually learn to open the door, and how does it compare to the existing rl_games PPO baseline?* — is unanswered and would need long runs on the Isaac Lab route, which is the only one that can currently reach TD3's warmup threshold at all.